# Fully dynamic light spanners in doubling metrics

Sujoy Bhore, Jonathan Conroy, Arnold Filtser

arxiv:2603.23490; CC BY 4.0

Editorial scope. The dynamic construction maintains delayed-greedy stretch and lightness invariants across insertions and deletions, then implements the local repair steps with multiscale coarse distance estimates and an induction controlling accumulated approximation error. All deletion and UpdateDistEstimates details from the full version are retained. Complete own-CCBY4 native author body retained; all support remains within this main task, no code rerun or correctness certification.

Allowed standard prerequisites (editorial).

- Standard doubling-metric packing, dynamic net trees and spanner lightness background. Original author statements and references retained. All source-specific local repair, delayed-greedy invariants and fast distance-estimation arguments included.

## 1 Introduction

Let $X \subset \mathcal{X}$ be a set of points in the metric $(\mathcal{X}, \delta_X)$. A graph $G = (X, E, \delta)$ with vertex set $X$ is a geometric graph if the weight function on the edges is the distance function $\delta$. In other words, the weight of each edge is the distance between its endpoints in $\mathcal{X}$. Observe that if $G$ is a geometric graph, then by the triangle inequality, for every $x, y \in X$, $\delta_G(x, y) \geq \delta(x, y)$. Given a parameter $t$, called the stretch, we say that $G$ is a (geometric) $t$-spanner for $X$ if $\delta_G(x, y) \leq t \cdot \delta(x, y)$ for every $x, y \in X$. The natural goal is

to construct, for a given stretch $t$, a $t$-spanner with small number of edges and a small total weight. Lightness and sparsity are two central measures for evaluating spanners. For a spanner $G = (X, E)$, the lightness is the ratio $w(G)/w(\text{MST})$ between the total weight of $G$ and the weight of a minimum spanning tree on $X$. The sparsity of $G$ is the ratio $|E(G)|/|E(\text{MST})| \approx |E(G)|/|X|$, comparing the number of edges of $G$ to that of an MST. Since every spanner (with finite stretch) is connected and therefore contains a spanning tree, the lightness and sparsity of $G$ constitute a measure of how close a spanner (specifically $w(G)$ and $|E(G)|$) to the optimal weight and the optimal number of edges of a spanner with finite stretch. A long line of research has studied spanners in Euclidean spaces [ADM$^+$95, RS98, HIS13, ES15, FN22, BT22, LS25], doubling [GGN04, CG09, Smi09, Got15, FS20, BLW19, BM25], planar and minor-free metric [BLWN17, GS02, CAFKL20, BKK$^+$25], and general graphs [PS89, ADD$^+$93, ES16, ENS15, CW18, FS20, LS23, Bod24]. Of particular interest, we focus on low-dimensional Euclidean space, and more generally on doubling metrics.

**Doubling metrics.** Let $B(x, r) = \{y | \delta(x, y) \leq r\}$ be a ball of radius $r \geq 0$ centered at a point $x \in \mathcal{X}$. A metric $(\mathcal{X}, \delta)$ has doubling dimension $d$ if for every $r > 0$, every ball of radius $r$ can be covered by at most $2^d$ balls of radius $r/2$: $\forall x \in X, r > 0, \exists Y \subseteq X : |Y| \leq 2^d$ and $B_X(x, r) \subseteq \cup_{y \in Y} B_X(y, r/2)$. The study of spanners in doubling metrics was initiated by Gao, Guibas, and Nguyen [GGN04], who proved that any $n$-point set in a metric space of doubling dimension $d$ admits a $(1 + \varepsilon)$-spanner with $\varepsilon^{-O(d)}n$ edges. Smid [Smi09], via an analysis of the greedy algorithm, showed that greedy $(1+\varepsilon)$-spanners have $\varepsilon^{-O(d)}n$ edges and achieve lightness $\varepsilon^{-O(d)} \log n$. While it has long been known that $d$-dimensional Euclidean point sets admit $(1+\varepsilon)$-spanners with lightness $\varepsilon^{-O(d)}$, it remained a major open problem whether an analogous result holds for metric spaces of low doubling dimension. In a major breakthrough, Gottlieb [Got15] proved that any metric space of doubling dimension $d$ admits a $(1 + \varepsilon)$-spanner with lightness $(d/\varepsilon)^{O(d)}$. Gottlieb devised a complicated spanner construction instead of the classic greedy spanner. Later, Filtser and Solomon [FS20] showed that the greedy spanner is existen-

tially optimal, concluding that it has lightness $(d/\varepsilon)^{O(d)}$ as well. Finally, Borradaile, Le, and Wulff-Nilsen [BLW19] showed the greedy spanner has weight $\varepsilon^{-O(d)}$, which is optimal.

**Dynamic Spanners.** The problem of maintaining a sparse $(1+\varepsilon)$-spanner for a point set $X$ under insertions and deletions has also been studied over the years. Already back in 1999, Arya et al. [AMS99] obtained polylogarithmic update time in a restricted model with random updates (a deletion removes a uniformly random point from $S$, and an insertion results in a uniformly random point in the updated set). Later, Bose et al. [BGM04] studied the problem in the context of $\theta$-graphs, and presented an algorithm that supports insertions only and achieves polylogarithmic update time. Gao et al. [GGN04] studied kinetic and dynamic geometric spanners for points in bounded doubling dimensions, and obtained a fully dynamic spanner with worst-case update time $O(\log \Phi)$, where $\Phi$ is the aspect ratio of the point set $X$, defined as the ratio between the maximum and minimum pairwise distances in $X$. Krauthgamer and Lee [KL04] obtained similar results for proximity search, where the time bounds of their dynamic updates depend also in the aspect ratio. For proximity search, later, Cole and Gottlieb [CG06] removed the dependency on the aspect ratio. For the Euclidean plane, Abam et al. [AdBG08] presented an algorithm with $O(\log n)$ update time. For doubling metrics, Roditty [Rod12] presented the first fully dynamic algorithm for maintaining a geometric $(1+\varepsilon)$-spanner, with update time depending only on the number of points in $X$. The algorithm obtained in [Rod12] supports insertions in amortized $O(\log n)$ time and deletions in amortized $\tilde{O}(n^{1/3})$ time where $\tilde{O}$ hides polylogarithmic factors, and maintains spanner with $O(n/\varepsilon^d)$ edges. Gotllieb and Roditty [GR08a] gave a dynamic spanner that supports insertions in $O\left(\varepsilon^{-O(d)} \log^2 n\right)$ amortized time and deletions in $O\left(\varepsilon^{-O(d)} \log^3 n\right)$ amortized time. Later, in [GR08b], [1] they gave a $(1+\varepsilon)$-spanner of constant degree that supports updates in $\varepsilon^{-O(d)} \log n$ worst-case time. Roughly speaking, these "classic" constructions of dynamic spanners for doubling metric are based on a net tree

[1] Gotllieb and Roditty [GR08b] additionally assume that they can query the distance between a deleted point and a non-deleted point, which (for example) is possible for Euclidean metrics.

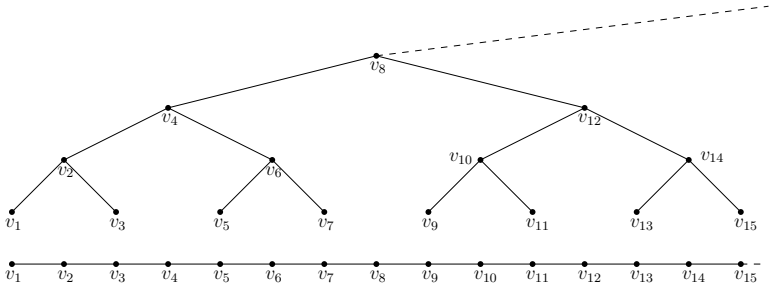



Figure 1: On bottom: The path graph $P_n$. All edges are unit weight. On top: a depiction of the net tree $T$ of $P_n$. There is a laminar hierarchy of nets; the $2^i$-net is $N_i = \{v_j \in P_n \mid j \mod 2^i = 0\}$ (all vertices at hight $i$ in the tree and above). The net-tree spanner contains all the tree edges, and many additional edges. The weight of the edge $\{v_i, v_j\}$ is $|i-j|$. The tree $T$ has depth $\log n$, and the total weight of the edges going from depth $i$ vertices to depth $i+1$ is $\Theta(n)$. Thus $w(T) = \Theta(n \cdot \log n)$. As the weight of the MST of $P_n$ is $n-1$, the net-tree spanner has lightness $\Omega(\log n)$.

(see Section 2). While it is straightforward to construct a sparse spanner from a net tree, this tool is too rough to obtain lightness. Indeed, in Figure 1 we illustrate the net tree of the path graph, and show that it has lightness $\Omega(\log n)$. A more recent approach to constructing dynamic sparse spanners is using locality-sensitive orderings (LSO) (introduced by Chan, Har-Peled, and Jones [CHJ20]). In Section A, we summarize the history of LSO, and prove that LSO-based spanners have logarithmic lightness (even for $\mathbb{R}^1$). While there is a substantial body of work on dynamic spanners in Euclidean metrics and, more generally, in doubling metrics, as summarized above, there is comparatively little work on dynamic light spanners. Controlling lightness is arguably a significantly harder objective, as evidenced by the extensive literature in the offline setting (see, e.g., [LS25, ADM+95, RS98, BLWN17, LS23, NS07, BM25, FS20, Got15, ES15, BT22, CCL+25] and the references therein).

**Question 1.1.** Can we design a dynamic algorithm that maintains a light-weight spanner for a point set under dynamic updates in doubling metrics with polylogarithmic update time?

Partial progress toward Question 1.1 was made by Eppstein and Khodabandeh [EK24], who showed that one can maintain a low-recourse light spanner for dynamic point sets in low-dimensional Euclidean space. The recourse of a dynamic spanner is the number of edges that change after each insertion or deletion. They obtained a construction with amortized $O(1)$ recourse per insertion and amortized $O(\log \Phi)$ recourse per deletion. However, their result addresses recourse rather than efficiency: the actual time to update the spanner can be very large (no faster than just recomputing a spanner from scratch), and therefore their work does not resolve the question of achieving polylogarithmic (or even sublinear) update time.

## 1.1 Our Contribution

We resolve Question 1.1 in the affirmative. We say a set of points $X$ is $(a, b)$-bounded if $\min_{u,v \in X} \delta(u, v) \geq a$ and $\max_{u,v \in X} \delta(u, v) \leq b$.

**Theorem 1.2.** Let $(\mathcal{X}, \delta)$ be a metric space with doubling dimension $d$, and let $\varepsilon > 0$ and $\Phi > 0$ be parameters. Let $X \subseteq \mathcal{X}$ be a set of points undergoing insertions and deletions, such that at all times $X$ is $(1, \Phi)$-bounded. We maintain a $(1 + \varepsilon)$-spanner $L$ of $X$ with lightness at most $\varepsilon^{-O(d)}$, where each insertion/deletion to $X$ can be processed in time $(\varepsilon^{-1} \cdot \log \Phi)^{O(d)}$.

For the sake of simplicity, we state our theorem as though the maximum distance $\Phi$ must be given in advance, and the minimum distance between points in $X$ must be at least 1. Using standard techniques [KL04, EK24] we could remove these restrictions at no loss in the running time: we obtain a data structure whose update time depends adaptively on the aspect ratio $\Phi$ of the current point set $X$. See Remark 4.8. Additionally, we remark that we do not need to know the doubling dimension $d$ in advance; the value $d$ only appears in the runtime analysis.

**Techniques.** Our proof begin somewhat similarly to Eppstein and Khodabandeh [EK24]: roughly speaking, we seek to maintain a greedy spanner on top of a dynamic sparse spanner of points $X$ (crucially, this spanner is much more structured than just the greedy spanner on the complete geometric graph on

$X$). In Section 3, we first give a new (and arguably much simpler[2]) algorithm and analysis that such a spanner can be maintained with bounded recourse. After each insertion/deletion, we identify $O(\log \Phi)$ edges $(u, v)$ of our greedy spanner $L$ which might have to change, to satisfy the greedy spanner invariants: roughly speaking, we examine the distance $\delta_{L<}(u, v)$ between $u$ and $v$ in the graph consisting of all of edges of $L$ that are shorter than $(u, v)$[3], and we insert or delete $(u, v)$ to $L$ based on whether $\delta_{L<}(u, v) \leq (1 + \varepsilon)\delta(u, v)$. Our key technical contribution (in Section 4) is to show that we can efficiently maintain estimates of $\delta_{L<}(u, v)$ even as we insert or delete edges into $L$.

## 1.2 Related Work

**Dynamic Graph Spanners.** In general graphs, the goal is to maintain, under edge updates, for any integer $k \geq 2$, a spanner with stretch $2k - 1$ and $\widetilde{O}(n^{1+1/k})$ edges. Assuming the Erdős–Girth conjecture, this trade-off is essentially optimal. The dynamic spanner problem was initiated by Ausiello et al. [AFI05], and has since been studied extensively; see, for example, [BKS12, BK16, Elk11, BFH21, FG19]. However, these works focus primarily on sparsity, and the resulting techniques do not naturally extend to maintaining light spanners. Moreover, due to inherent degree barriers in general graphs, this line of work exclusively considers edge updates. In contrast, in geometric and other structured metric spaces, vertex updates are often the more natural model, since the underlying graph need not be maintained explicitly. This distinction is crucial for light spanners, whose construction and analysis rely fundamentally on geometric structure rather than purely combinatorial sparsification.

[2]We avoid the bucketing technique in the algorithm of [EK24] and instead work with all spanner edges at once; we maintain spanner invariants that work in doubling metrics and not just Euclidean metrics (unlike the leapfrog property used in [EK24]); and finally, we give a simple worst-case bound on the recourse of $O(\log \Phi)$, rather than using a potential function to bound the amortized recourse.

[3]Our description here assumes we want to maintain a greedy spanner, but actually we maintain a slightly more robust type of spanner which we call the delayed greedy spanner: we work with distances $\delta^*(u, v)$ rather than $\delta_{L<}(u, v)$, as defined in Section 2.2. We do this to control cascading changes to $L$.

**Online Spanners.** Spanners have also been studied extensively in the closely related online model. In this setting, points from a metric space arrive one by one, and at each step, the algorithm is given the subset of points that arrived so far. The goal is to maintain a $t$-spanner at all times. Unlike in the fully dynamic setting, the algorithm is allowed to add edges when a new point arrives, but may never remove previously added edges. In addition, the total number of points $n$ is not known in advance. A number of algorithms and lower bounds have been obtained for this model, which addressed both sparsity and lightness guarantees along with stretch, under online constraints; see, e.g., [BFKT24, BT25, AAA+06, AA92, BC97, HLP17, NPS11]. These results achieve near-optimal trade-offs in the online regime and highlight the fundamental differences between incremental constructions and fully dynamic settings.

# 2 Net trees and greedy spanners

**Preliminaries.** Throughout this paper, let $M = (\mathcal{X}, \delta)$ be a metric space with constant doubling dimension $d = O(1)$. For any vertex $x \in \mathcal{X}$ and radius $r > 0$, we let $B(x, r)$ denote the set of points within distance $r$ of $x$; that is, $B(x, r) \coloneqq \{y \in \mathcal{X} : \delta(x, y) \leq r\}$. If $G$ is a (geometric) graph, we use the notation $\delta_G(\cdot, \cdot)$ to denote shortest-path distances on $G$. For any positive integer $n$, we write $[n]$ to mean $\{i \in \mathbb{Z} : 0 \leq i \leq n\}$. Throughout this paper, we fix some sufficiently small positive integer $\varepsilon > 0$ and aim to construct a light $(1+O(\varepsilon))$-spanner; our Theorem 1.2 will follow after rescaling $\varepsilon \leftarrow \varepsilon/O(1)$.

## 2.1 Net-Tree Spanners

In this subsection, we review the classic net-tree spanner [GGN04].

**Net tree.** Let $X \subseteq \mathcal{X}$ be a set of points. Assume that the smallest distance between points in $X$ is $1$, and the largest distance is $\Phi$; for simplicity we assume $\Phi = 2^i$ for some integer $i$. A $\Delta$-net of $X$ is a set of points $N$ such that every point $x \in X$ is within distance $\Delta$ of some point in $N$, and any two net points in $N$ are at distance

greater than $\Delta$. A $\Delta$-net can be constructed greedily. A net hierarchy of $X$ is a hierarchical sequence of nets $(N_0, N_1, \ldots, N_{\log \Phi})$ such that every $N_i$ is a subset of $N_{i-1}$, and $N_i$ is a $2^i$-net of $N_{i-1}$; we take $N_0 = X$. Observe that (by a geometric series) every point in $X$ is within distance $2^{i+1}$ of some net point in $N_i$. Moreover, the packing bound of doubling metrics implies that there are not too many points close together in each $\Delta$-net.

**Observation 2.1** (Packing bound)**.** Let $N$ be a set of points that are pairwise at distance at least $\Delta$. For any $\alpha \geq 1$ and $x \in \mathcal{X}$, there are at most $\alpha^{O(d)}$ points in $B(x, \alpha \cdot \Delta) \cap N$.

An implicit representation of the net hierarchy $\mathcal{N}$ can be maintained using a data structure called the net tree [GGN04, KL04, HPM06]. We will sometimes abuse terminology and call $\mathcal{N}$ a net tree.

**Net-tree spanner.** The net-tree spanner was introduced (under the name "deformable spanner") by Gao, Guibas, and Nguyen [GGN04]; similar ideas were independently designed by Krauthgamer and Lee [KL04]. The $\varepsilon$-net-tree spanner of $X$ with respect to the net tree $\mathcal{N} = (N_0, \ldots, N_{\log \Phi})$ is the graph with vertex set $X$ and with the following edge set: for every scale $i \in [\log \Phi]$, add an edge between any two vertices $u, v \in N_i$ if $\delta(u, v) \leq c \cdot 2^i$, where $c \coloneqq 4 + 16\varepsilon^{-1}$.

**Lemma 2.2** ([GGN04, Theorem 3.2])**.** An $\varepsilon$-net-tree spanner of $X$ is a $(1 + \varepsilon)$-spanner of $X$.

For any pair of vertices $(u, v)$, we say the scale of $(u, v)$ is the value $i$ such that $\delta(u, v) \in [2^{i-1}, 2^i)$. Observe that an edge $(u, v)$ at scale $i$ in an $\varepsilon$-net-tree spanner has both endpoints in the $O(\varepsilon) \cdot 2^i$-net of the net tree (not the $2^i$-net). The packing bound implies that every vertex in the $\varepsilon$-net-tree spanner has degree at most $\varepsilon^{-O(d)} \cdot \log \Phi$. In fact, using a charging argument, one can show that the $\varepsilon$-net-tree spanner contains at most $\varepsilon^{-O(d)} \cdot n$ edges: that is, it is sparse. We frequently denote the $\varepsilon$-net-tree spanner as $S$.

**Dynamic maintenance.** Gao, Guibas, and Nguyen [GGN04] showed that the net tree spanner can be maintained dynamically. We summarize their guarantee below.

**Lemma 2.3** (Rephrasing of [GGN04, Theorem 4.2])**.** There is a data structure that implicitly maintains a hierarchy of nets $\mathcal{N} = (N_0, N_1, \ldots, N_{\log \Phi})$ of a point set $X$ undergoing insertions and deletions. Each insertion or deletion can be processed in $O(\log \Phi)$ time. Moreover[4], for any constant $\varepsilon > 0$, one can maintain an $\varepsilon$-net-tree spanner of $X$ with respect to $\mathcal{N}$, in $O(\log \Phi)$ time per insertion or deletion.

We will make use of Lemma 2.3, but we will have to slightly unwrap the black box — we will need some control on how the net tree $\mathcal{N}$ changes after each insertion or deletion. Fortunately, [GGN04] update the net tree $\mathcal{N}$ in a simple and structured way. Inserting a point $x$ into $X$ has the following result, described in Figure 2. Deleting a point $x$ from $X$ is described in Figure 3. We

$i_0 \leftarrow$ smallest scale such that the scale-$i_0$ net $N_{i_0} \in \mathcal{N}$ satisfies $\delta(x, N_{i_0}) < 2^{i_0}$
update $\mathcal{N}$ by inserting $x$ into net $N_i \in \mathcal{N}$ for all $i < i_0$

Figure 2: Pseudocode for [GGN04] algorithm to update net $\mathcal{N}$ after inserting point $x$ into $X$

update $\mathcal{N}$ by deleting $x$ from every net $N_i \in \mathcal{N}$
for every scale $i \leftarrow 1, 2, \ldots, \log \Phi$:
  while there exists some point $y \in N_{i-1}$ with $\delta(u, N_i) > 2^i$, add $y$ to $N_i$

Figure 3: Pseudocode for [GGN04] algorithm to update net $\mathcal{N}$ after deleting point $x$ from $X$

will not discuss the details of the data structures that [GGN04] maintain to implement these two operations efficiently. We have unwrapped the [GGN04] black box so that we can prove that every insertion/deletion of a point $x$ only changes $\mathcal{N}$ "locally" nearby $x$.

[4]Note that the phrasing of [GGN04, Theorem 4.2] does not explicitly separate the maintenance of the net hierarchy $\mathcal{N}$ from the maintenance of the spanner (they just state the existence of a dynamic spanner); however, our phrasing of Lemma 2.3 is immediate from their proof. In our application, we will need to dynamically maintain an $\varepsilon_1$-net-tree spanner of $X$ and an $\varepsilon_2$-net-tree spanner of $X$ (for two different values $\varepsilon_1 \neq \varepsilon_2$) with respect to the same net tree $\mathcal{N}$. This is possible using the data structure of [GGN04].

**Lemma 2.4.** Let $\mathcal{N}^{\text{old}} = (N_0^{\text{old}}, \ldots, \mathcal{N}_{\log \Phi}^{\text{old}})$ be a net tree for point set $X$. Let $\mathcal{N} = (N_0, \ldots, N_{\log \Phi})$ be the net tree produced from $\mathcal{N}^{\text{old}}$ by the [GGN04] algorithm after a point $x$ is inserted into (resp. deleted from) $X$. For any scale $i \in [\log \Phi]$, every point $y$ in the symmetric difference of $N_i$ and $N_i^{\text{old}}$ satisfies $\delta(y, x) \leq 2^i$.

Proof. When $x$ is inserted into $X$ (according to the pseudocode in Figure 2), the lemma follows trivially from the [GGN04] algorithm description: the only point that could differ between $N_i$ and $N_i^{\text{old}}$ is $x$. We now argue that the lemma holds when $x$ is deleted from $X$ (according to the pseudocode in Figure 3). By construction, the only point that could be removed from $N_i^{\text{old}}$ is $x$; that is, the set $N_i^{\text{old}} \setminus N_i$ is either $\varnothing$ or $\{x\}$. So it remains to show that every point $y$ in $N_i \setminus N_i^{\text{old}}$ satisfies $\delta(y, x) \leq 2^i$. The proof is by induction on $i$. When $i = 0$, every point in $\texttt{DS}.X$ is contained in $N_0^{\text{old}}$, so $N_0 \setminus N_0^{\text{old}} = \varnothing$ and the claim holds trivially. Now consider $i > 0$. Consider some point $y \in N_i \setminus N_i^{\text{old}}$. By the algorithm description, the point $y$ satisfies

$$\delta(y, N_i^{\text{old}} \setminus \{x\}) > 2^i.$$

There are two cases. In the first case, suppose $y \in N_{i-1}^{\text{old}}$. Because $N_i^{\text{old}}$ is a $2^i$-net for $N_{i-1}^{\text{old}}$, we have $\delta(y, N_i^{\text{old}}) \leq 2^i$. We conclude that $\delta(y, x) \leq 2^i$ as desired. In the second case, suppose $y \notin N_{i-1}^{\text{old}}$. Thus, $y \in N_{i-1} \setminus N_{i-1}^{\text{old}}$, and by induction we have that $\delta(y, x) \leq 2^{i-1} < 2^i$ as desired. □

Finally, we need one more guarantee from the [GGN04] data structure, which appears (implicitly) in the proof of [GGN04, Theorem 4.2].

**Claim 2.5** ([GGN04])**.** Consider the data structure of [GGN04] that maintains a hierarchy of nets $(N_0, N_1, \ldots, N_{\log \Phi})$ of $X$ and an $\varepsilon$-net-tree spanner. Let $c_\varepsilon = 4 + 16\varepsilon^{-1}$. Given a scale $i \in [\log \Phi]$ and a query point $x \in \mathcal{X}$, one can find all vertices in $N_i \cap B(x, c_\varepsilon \cdot 2^i)$ in time $\varepsilon^{-O(d)} = O(1)$.

In our applications, we will want something a bit more general: we fix a sufficiently large constant $\alpha$ (say, $\alpha > 100$ suffices) and we want to find all vertices in $N_i \cap B(x, \alpha \cdot c_\varepsilon \cdot 2^i)$ in time $\varepsilon^{-O(d)}$. Observe that Claim 2.5 allows us to do this: indeed, we could

simply maintain the data structure for an $\varepsilon/8$-net-tree spanner, which (by Claim 2.5) allows us to quickly find all vertices in $N_i \cap B(x, c_{\varepsilon/8} \cdot 2^i) \supseteq N_i \cap B(x, 100 \cdot c_\varepsilon \cdot 2^i)$.

**Corollary 2.6.** Consider the data structure of [GGN04] that maintains a hierarchy of nets $(N_0, N_1, \ldots, N_{\log \Phi})$ of $X$ and an $\varepsilon$-net-tree spanner. Let $c_\varepsilon = 4 + 16\varepsilon^{-1}$. Given a scale $i \in [\log \Phi]$ and a query point $x \in \mathcal{X}$, one can find all vertices in $N_i \cap B(x, 10 \cdot c_\varepsilon \cdot 2^i)$ in time $\varepsilon^{-O(d)} = O(1)$.

## 2.2 Invariants for Our Light Spanner: Delayed Greedy Spanner

While the net-tree spanner $S$ is sparse and can be maintained dynamically, it could have lightness $\Omega(\log n)$ (see Figure 1). We aim to find a subgraph of $S$ with $O(1)$ lightness. Intuitively, we want to maintain a greedy $(1+\varepsilon)$-spanner $H$ of the graph $S$: the greedy spanner (and even the approximate greedy spanner) are light [BLW19, FS20]. Recall that the greedy spanner initializes $H \leftarrow \varnothing$ and processes edges in $S$ from smallest to largest; when we process $(u, v)$, we add $(u, v)$ to $H$ iff $\delta_H(u, v) > (1+\varepsilon)\delta_S(u, v)$. In other words, the greedy spanner satisfies: for every edge $(u, v)$ in $S$, letting $H_{<(u,v)}$ denote the set of edges in $H$ with smaller[5] lengths than $(u, v)$, we have $\delta_{H_{<(u,v)}}(u, v) > (1+\varepsilon)\delta_S(u, v)$ iff $(u, v) \in L$.

**Delayed greedy spanner.** We maintain a light spanner, denoted $S \subseteq S$ that doesn't obey the greedy spanner invariants exactly; rather, we define a more robust set of invariants. For any scale $i \in [\log \Phi]$, we define the set of edges $L[i] \subseteq L$ to be the set of all edges in $L$ with scale strictly less than $i$; that is, every edge in $L[i]$ has length at most $2^{i-1}$. For any edge $(u, v)$ of the $\varepsilon$-net-tree spanner at scale $i$, we define $\delta^*(u, v) := \delta_{L[i]}(u, v)$; recall that $\delta_{L[i]}(\cdot, \cdot)$ denotes the shortest-path metric in the geometric graph induced by $L[i]$. To construct our light spanner, we maintain a subset of edges $L \subseteq S$ with the following properties:

**Invariant 2.7.** Every edge $(u, v) \in S \setminus L$ satisfies $\delta^*(u, v) \leq (1+\varepsilon) \cdot \delta(u, v)$.

[5] assume all edge lengths in $S$ are distinct, by breaking ties arbitrarily

**Invariant 2.8.** Every edge $(u,v) \in L$ satisfies $\delta^*(u,v) > (1+\frac{\varepsilon}{3}) \cdot \delta(u,v)$.

If $L$ satisfies Invariants 2.7 and 2.8, we call $L$ a delayed greedy spanner of $S$ (because there is a delay between the time an edge is added to the spanner and the time it is taken into account when making the next decision). We emphasize that the delayed greedy spanner invariants are different than the greedy spanner invariants for $H$: in our invariants, the choice of adding $(u,v)$ to the spanner depends on $L[i]$ (the set of edges in $L$ with scale strictly smaller than the scale of $(u,v)$), rather than $H_{<(u,v)}$ (the set of edges in the greedy spanner $H$ that are strictly shorter than $\delta(u,v)$). Nevertheless, we show that any set of edges $L$ that satisfies Invariants 2.7 and 2.8 is a $(1+O(\varepsilon))$-spanner (Lemma 2.9) and has constant lightness (Lemma 2.10).

**Lemma 2.9.** Let $S$ be an $\varepsilon$-net-tree spanner for $X$, and let $L \subseteq S$. If Invariant 2.7 holds, then $L$ is a $(1+3\varepsilon)$-approximate spanner for $(X,\delta)$.

Proof. By Lemma 2.2, the $\varepsilon$-net-tree spanner $S$ is a $(1+\varepsilon)$-spanner for $(X,\delta)$. Moreover, Invariant 2.7 implies that $L$ is a $(1+\varepsilon)$-spanner for $S$: by triangle inequality it suffices to check that every edge of $S$ is preserved up to $1+\varepsilon$ approximation in $L$, and indeed, for every edge $(u,v) \in S$, we have $\delta_L(u,v) \le \delta^*(u,v) \le (1+\varepsilon)\delta(u,v) \le (1+\varepsilon)\delta_S(u,v)$. We conclude that $L$ is a spanner for $(X,\delta)$ with stretch $(1+\varepsilon)^2 \le (1+3\varepsilon)$. □

**Lemma 2.10.** Let $S$ be an $\varepsilon$-net-tree spanner for $X$, and let $L \subseteq S$. If Invariants 2.7 and 2.8 hold, then $L$ has lightness $\varepsilon^{-O(d)}$.

Proof. We use, as a black box, the fact that the greedy $(1+\frac{\varepsilon}{3})$-spanner in a metric space of doubling dimension $d$ has lightness $\varepsilon^{-O(d)}$ [BLW19]. We consider the shortest-path metric $\delta_L$ induced by the edges of $L$; because $(X,\delta)$ has doubling dimension $d$, the stretch bound of Lemma 2.9 together with the definition of doubling dimension implies that the doubling dimension of $(X,\delta_L)$ is $O(d)$ (see [FS20, Observation 7]). Consider building the greedy $(1+\frac{\varepsilon}{3})$-spanner $\hat{L}$ of $L$: that is, we initialize $\hat{L} \leftarrow \varnothing$, then iterate over the edges of $L$ from smallest to largest[6], and add an edge

[6] by triangle inequality, it suffices to examine only the edges of $L$ when building the greedy spanner, rather than all pairs of vertices in $L$

$(u,v) \in L$ into the spanner $\hat{L}$ iff $\delta_{\hat{L}}(u,v) > (1+\frac{\varepsilon}{3})\delta_L(u,v)$. The greedy spanner $\hat{L}$ has lightness $\varepsilon^{-O(d)}$. We now charge every edge of $L$ to an edge of $\hat{L}$. If an edge $(u,v) \in L$ is added to $\hat{L}$, then charge $(u,v)$ to itself. Otherwise, suppose $(u,v) \in L$ is not added to $\hat{L}$. This means that there is a path $P$ between $u$ and $v$ in $\hat{L}$ (at the instant that $(u,v)$ is considered) with length at most $(1+\frac{\varepsilon}{3})\delta_L(u,v)$. Because $S$ is a geometric graph, $\delta_S(u,v) = \delta(u,v)$; i.e., $P$ has length at most $(1+\frac{\varepsilon}{3})\delta(u,v)$. Let $i$ be the scale of $(u,v)$, meaning that $\delta(u,v) \in [2^{i-1}, 2^i)$. The path $P$ contains some scale-$i$ edge $(u',v')$ in the greedy spanner $\hat{L}$: otherwise, if $P$ contained only scale-$j$ edges with $j < i$, we would have $\delta^*(u,v) \leq (1+\frac{\varepsilon}{3}) \cdot \delta(u,v)$, contradicting Invariant 2.8. Charge $(u,v)$ to the scale-$i$ edge $(u',v')$. We now claim that every edge $(u',v')$ in $\hat{L}$ is charged at most $\varepsilon^{-O(d)}$ times. Indeed, whenever a scale-$i$ edge $(u',v')$ is charged by a scale-$i$ edge $(u,v)$, we have $\delta(u,u') \leq (1+\frac{\varepsilon}{3})2^i$; this is because, by definition of charging, $u'$ lies on a path $u$ and $v$ with length at most $(1+\frac{\varepsilon}{3})\delta(u,v) \leq (1+\frac{\varepsilon}{3})2^i$. The packing bound (Observation 2.1) implies that there are only $\varepsilon^{-O(d)}$ scale-$i$ edges that could charge $(u',v')$. Because we only charge scale-$i$ edges in $L$ to scale-$i$ edges in $\hat{L}$, we conclude that $L$ is at most $\varepsilon^{-O(d)}$ times heavier than $L$, which is $\varepsilon^{-O(d)}$ times heavier than the minimum spanning tree on $X$. □

# 3 Light Spanner with Worst-Case Recourse Bounds

We now describe how to maintain our light spanner $L$, satisfying Invariants 2.7 and 2.8. The current section focuses on describing a spanner with a small recourse bound, and proving that our updates maintain Invariants 2.7 and 2.8. In Section 4, we explain how to perform these updates quickly. We maintain a data structure *DS*. It stores:

- a set of points *DS*.$X$
- a net-tree *DS*.$\mathcal{N}$ on the points DS.$X$
- an $\varepsilon$-net-tree spanner *DS*.$S$ of DS.$X$ with respect to DS.$\mathcal{N}$

- a subgraph $\mathtt{DS}.L \subseteq \mathtt{DS}.S$ called the light spanner, such that Invariants 2.7 and 2.8 hold.

All these variables are initialized as $\varnothing$. The data structure supports two operations—insertion and deletion of a point $x$. We say that a procedure Insert$(x)$ correctly implements insertion of point $x$ if the procedure assigns $\mathtt{DS}.X \leftarrow \mathtt{DS}.X \cup \{x\}$ and updates $\mathtt{DS}.\mathcal{N}$, $\mathtt{DS}.S$, and $\mathtt{DS}.L$ to satisfy the descriptions above. Similarly, a procedure Delete$(x)$ correctly implement deletion of a point $x$ if it assigns $\mathtt{DS}.X \leftarrow \mathtt{DS}.X \setminus \{x\}$ and updates $\mathtt{DS}.\mathcal{N}$, $\mathtt{DS}.S$, and $\mathtt{DS}.L$.

## 3.1 Insertion

To insert a point $x$, we first update the net-tree according to [GGN04] (as summarized in our Lemma 2.3) and then recompute the spanner edges $\mathtt{DS}.L$ nearby $x$ to satisfy the invariants. The pseudocode is given below. Our key insight is that when we recompute spanner edges at level $i$, we only need to recompute edges with endpoints in $B(x, O(2^i))$.

```
DS.Insert(x):
  DS.X ← DS.X ∪ {x}
  update DS.N and DS.S according to Lemma 2.3
  DS.Recompute(x)

DS.Recompute(x):
  for every scale i ← 0, 1, ..., log Φ:
      for every scale-i edge (u, v) of DS.S with u, v ∈ B(x, 4 · 2^i):
          if δ*(u, v) > (1 + ε) · δ(u, v):
              DS.L ← DS.L ∪ {(u, v)}
          else:
              DS.L ← DS.L \ {(u, v)}
```

**Lemma 3.1.** The procedure $\mathtt{DS}$.Insert$(x)$ correctly implements the insertion of a point $x$.

Proof. Let $\mathcal{N}^{\text{old}}$, $S^{\text{old}}$, $L^{\text{old}}$, and $X^{\text{old}}$ denote $\mathtt{DS}.\mathcal{N}$, $\mathtt{DS}.S$, $\mathtt{DS}.L$, and $\mathtt{DS}.X$ (respectively) before the insertion process. For brevity, we write $\mathcal{N}$, $S$, $L$, and $X$ instead of $\mathtt{DS}.\mathcal{N}$, $\mathtt{DS}.S$, $\mathtt{DS}.L$, and $\mathtt{DS}.X$. By Lemma 2.3, the variables $\mathtt{DS}.X$, $\mathtt{DS}.\mathcal{N}$ and $\mathtt{DS}.S$ are correctly maintained. Moreover, we have that $L \subseteq S$, because (by description of the algorithm) $S^{\text{old}} \subseteq S$ and (by assumption) $L^{\text{old}} \subseteq S^{\text{old}}$. It

remains to show that that every edge $(u, v)$ in $S$ satisfies Invariants 2.7 and 2.8. Suppose that edge $(u, v)$ is at scale $i$ (that is, $\delta(u, v) \in (2^{i-1}, 2^i]$). Note that the distance $\delta^*(u, v) = \delta_{L[i]}(u, v)$ depends only on the edges in $L$ with scale strictly less than $i$; thus, it suffices to show that $(u, v)$ satisfies the two invariants at the moment after the for-loop processes scale $i$. If $u, v \in B(x, 4 \cdot 2^i)$, then $(u, v)$ satisfies the two invariants by description of the algorithm. Now suppose that some endpoint, without loss of generality $u$, is not in $B(x, 4 \cdot 2^i)$. In this case, $(u, v)$ is in $L$ if and only if $(u, v)$ is in $L^{\text{old}}$. There are two cases:

- **Suppose that $(u, v)$ is not in $L$.** Thus $(u, v)$ is not in $L^{\text{old}}$. To satisfy Invariant 2.7, we must show $\delta_{L[i]}(u, v) \leq (1 + \varepsilon) \cdot \delta(u, v)$. In fact, we show that either $\delta_{L[i]}(u, v) \leq \delta_{L^{\text{old}}[i]}(u, v)$ or $\delta_{L^{\text{old}}[i]}(u, v) > 2\delta(u, v)$; see Figure 4. This suffices, as $L^{\text{old}}$ satisfies Invariant 2.7: we conclude that $\delta_{L[i]}(u, v) \leq \delta_{L^{\text{old}}[i]}(u, v) \leq (1 + \varepsilon)\delta(u, v)$ as desired. Consider a shortest path $P$ in $L^{\text{old}}[i]$ between $u$ and $v$. The description of the algorithm implies that the only edges that that change between $L[i]$ and $L^{\text{old}}[i]$ are in the ball $B(x, 4 \cdot 2^{i-1})$. Because we assumed $\delta(u, x) > 4 \cdot 2^i$, triangle inequality implies that endpoints of any edge $(u', v')$ in $L^{\text{old}}[i] \setminus L[i]$ are far from $u$: specifically, $\delta(u, u') \geq 4 \cdot 2^i - 4 \cdot 2^{i-1} = 2 \cdot 2^i$, and so $\delta_{L^{\text{old}}}(u, u') \geq 2 \cdot 2^i$. If $P$ has length $\geq 2 \cdot 2^i$ then we are done; otherwise, $P$ does not include any edges in $L^{\text{old}}[i] \setminus L[i]$, and so $P$ is also a path in $L[i]$ as desired.

- **Suppose $(u, v)$ is in $L$.** Thus $(u, v)$ is in $L^{\text{old}}$. To satisfy Invariant 2.8, we must show that $\delta_{L[i]}(u, v) > (1 + \frac{\varepsilon}{3}) \cdot \delta(u, v)$. We claim that either $\delta_{L[i]}(u, v) \geq \delta_{L^{\text{old}}[i]}(u, v)$, or $\delta_{L[i]}(u, v) \geq 2\delta(u, v)$. This suffices to prove the claim, because $L^{\text{old}}$ satisfies Invariant 2.8 and so $\delta_{L^{\text{old}}[i]}(u, v) > (1 + \frac{\varepsilon}{3})\delta(u, v)$; thus, in either case, $\delta_{L[i]}(u, v) > (1 + \frac{\varepsilon}{3})\delta(u, v)$. Let $P$ be a shortest path between $u$ and $v$ in $L[i]$. If $\delta_{L[i]}(u, v) \geq \delta_{L^{\text{old}}[i]}(u, v)$ then we are done. Otherwise, if $\delta_{L[i]}(u, v) < \delta_{L^{\text{old}}[i]}(u, v)$, then $P$ must contain some edge of $L[i]$ that is not in $L^{\text{old}}[i]$. But the description of the algorithm implies that any such edge has endpoints in $B(x, 4 \cdot 2^{i-1})$. By triangle inequality and our assumption that $\delta(u, x) \geq 4 \cdot 2^i$, we have $\delta(u, u') \geq 2 \cdot 2^i$, and so $P$ has length at least $2 \cdot 2^i$ as desired. □

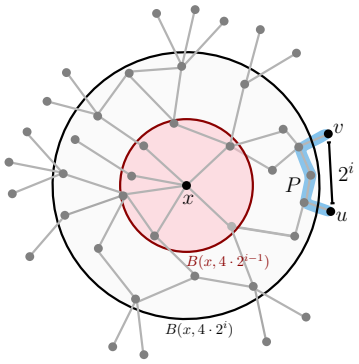



Figure 4: A light spanner $L^{\text{old}}$. For a given scale $i$, the spanners $L[i]$ and $L^{\text{old}}[i]$ differ only within the ball $B(x, 4 \cdot 2^{i-1})$. A scale-$i$ pair $(u, v)$ outside of $B(x, 4 \cdot 2^i)$. A short path $P$ between $u$ and $v$ in $L^{\text{old}}$ does not wander inside $B(x, 4 \cdot 2^{i-1})$, so $P$ is also in $L$.

We remark that the Insert procedure has bounded recourse: at most $\varepsilon^{-O(d)} \log \Phi$ edges change in the spanner $L$. This is because, for every scale $i \in \lceil \log \Phi \rceil$, there are at most $\varepsilon^{-O(d)}$ scale-$i$ edges in the $\varepsilon$-net-tree spanner in $B(x, 4 \cdot 2^i)$, by definition of net-tree spanner and packing bound (Observation 2.1).

## 3.2 Deletion

The procedure for deleting a point $x$ is similar to insertion. The procedure *DS*.Delete($x$) first updates the net tree in a standard manner by greedily fixing each net [GGN04, KL04], then deletes all edges in $L$ that were incident to $x$, and finally recompute the spanner edges nearby $x$ using the DS.Recompute procedure. The pseudocode and analysis are fairly similar to insertion procedure (albeit slightly more complicated). Like insertion, the deletion procedure has recourse at most $\varepsilon^{-O(d)} \cdot \log \Phi$. The pseudocode for the DS.Delete procedure is given in Figure 5.

**Lemma 3.2.** The procedure *DS*.Delete($x$) correctly implements the deletion of a point $x$.

Proof. The proof is very similar to that of Lemma 3.1. It is immediate from the description of the algorithm and Lemma 2.3 that

DS.Delete($x$):
  DS.$X \leftarrow$ DS.$X \setminus \{x\}$
  update DS.$\mathcal{N}$ and DS.$S$ according to Lemma 2.3
  delete every edge in DS.$L$ incident to $x$
  DS.Recompute($x$)

Figure 5: Deletion procedure with bounded recourse

that DS.$\mathcal{N}$ remains a valid net tree for DS.$X$ after the deletion procedure, and DS.$S$ is still an $\varepsilon$-net-tree spanner of DS.$\mathcal{N}$. Let $\mathcal{N}^{\text{old}}$, $S^{\text{old}}$, and $L^{\text{old}}$ denote DS.$\mathcal{N}$, DS.$S$, and DS.$L$ (respectively) before the deletion process. We write $\mathcal{N}$, $S$, and $L$ instead of DS.$\mathcal{N}$, DS.$S$, and DS.$L$. First we observe that $L \subseteq S$. Indeed, if $(u, v)$ is an edge added to $L$ by DS.Recompute($x$), then $(u, v)$ is in $S$ and we are done. Otherwise, $(u, v)$ is in $L^{\text{old}}$ and thus is in $S^{\text{old}}$. In this case we are not immediately done, because we no longer have the property that $S^{\text{old}} \subseteq S$; nevertheless, observe that the only edges in $S^{\text{old}} \setminus S$ are edges incident to $x$. Moreover, $(u, v)$ is some scale-$i$ edge with one endpoint not in $B(x, 4 \cdot 2^i)$, meaning $(u, v)$ is not incident to $x$. Thus $(u, v)$ is in $S$. Next we show that every scale-$i$ edge $(u, v)$ in $S$ satisfies Invariants 2.7 and 2.8. If $(u, v)$ has both endpoints in $B(x, 4 \cdot 2^i)$, then it is processed by the call to DS.Recompute($x$) and thus satisfies the invariants. So now suppose that $\delta(u, x) > 4 \cdot 2^i$. There are two cases.

- **Suppose $(u, v)$ is not in $L$.** In this case, the description of the algorithm implies that $(u, v)$ is not in $L^{\text{old}}$. This means that either $(u, v)$ is in $S^{\text{old}} \setminus L^{\text{old}}$ or $(u, v)$ was not an edge in $S^{\text{old}}$. We claim (and prove below) that the only edges in $S \setminus S^{\text{old}}$ at scale $i$ have both endpoints in $B(x, 4 \cdot 2^i)$. This implies that $(u, v)$ was not an edge in $S^{\text{old}}$. From here, the proof is exactly the same as in Lemma 3.1, and we repeat it here. We show that $\delta_{L[i]}(u, v) \leq \delta_{L^{\text{old}}[i]}(u, v) \leq (1 + \varepsilon)\delta(u, v)$. Indeed, the shortest path $P$ in $L_i^{\text{old}}$ between $u$ and $v$ has length at most $(1+\varepsilon)\delta(u, v)$ by Invariant 2.7, and this path also appears in $L_i$, because the only edges that change between $L_i^{\text{old}}$ and $L_i$ are in $B(x, 4 \cdot 2^{i-1})$ and thus are at least distance $2^i$ away from $u$. It remains to show that if $(u, v) \in S \setminus S^{\text{old}}$ then $u, v \in B(x, 4 \cdot 2^i)$. Because $(u, v)$ is a scale-$i$ edge of $S$, the endpoints $u$ and $v$ belong to $\varepsilon 2^i$-net of the net-tree DS.$\mathcal{N}$. In other words, letting

$i' := \log(\varepsilon 2^i)$, both $u$ and $v$ belong to the net $N_{i'}$. Moreover, because $(u, v)$ is not in $S^{\text{old}}$, one endpoint (WLOG $u$) is $N_{i'}^{\text{old}}$ but not $N_{i'}$. By Lemma 2.4, we have $\delta(u, x) \leq 2^{i'}$. By triangle inequality, $\delta(v, x) \leq 2^{i'} + 2^i < 2 \cdot 2^i$. We conclude that $u, v \in B(x, 4 \cdot 2^i)$ as desired.

- **Suppose $(u, v)$ is in $L$.** This case is exactly the same as in the proof of Lemma 3.1, and we repeat it. We show that either $\delta_{L[i]}(u, v) \geq \delta_{L^{\text{old}}[i]}(u, v)$ or $\delta_{L[i]}(u, v) > 2\delta(u, v)$. If $\delta_{L^{\text{old}}[i]}(u, v) \geq \delta_{L[i]}(u, v)$, we are done because $(u, v)$ is in $L^{\text{old}}$ and (by Invariant 2.7) we have $\delta_{L^{\text{old}}[i]}(u, v) > (1 + \frac{\varepsilon}{3})\delta(u, v)$. Otherwise, because the only edges that change between $L^{\text{old}}[i]$ and $L[i]$ lie in $B(x, 4 \cdot 2^{i-1})$, we conclude that the shortest path between $u$ and $v$ must have length at least $2\delta(u, v)$. Either way, $(u, v)$ satisfies Invariant 2.7. □

Finally, we pause to remark that in our proof of Lemma 3.1 (and also Lemma 3.2), we have actually proved the following claim, which will be helpful later.

**Lemma 3.3.** Consider the data structure *DS* before and after running *DS*.Insert$(x)$ or *DS*.Delete$(x)$. Let $S^{\text{old}}$ (resp. $L^{\text{old}}$) denote *DS*.$S$ (resp. *DS*.$L$) before the insertion/deletion is performed. Let $(u, v)$ be a scale-$i$ edge in *DS*.$S$, with some endpoint outside of $B(x, 4 \cdot 2^i)$. Then $(u, v)$ is an edge in $S^{\text{old}}$, and either (1) $\delta_{DS.L[i]}(u, v) = \delta_{L^{\text{old}}[i]}(u, v)$, or (2) both $\delta_{DS.L[i]}(u, v)$ and $\delta_{L^{\text{old}}[i]}(u, v)$ are larger than $2\delta(u, v)$.

# 4 Implementing Updates Quickly

In this section, we show how to modify the data structure so that the DS.Insert and DS.Delete procedures can be executed quickly. By prior work on the net-tree spanner (Lemma 2.3), the net-tree DS.$\mathcal{N}$ and the $\varepsilon$-net-tree spanner DS.$S$ can be maintained in $O(\log \Phi)$ time. The DS.Recompute procedure iterates over each of the $\log \Phi$ scales. For each scale $i$, it examines the $\varepsilon^{-O(d)} \log \Phi$ many scale-$i$ edges in $B(x, 4 \cdot 2^i)$; by Corollary 2.6 these edges can be found in $\varepsilon^{-O(d)} = O(1)$ time. Each examined edge $(u, v)$ is added or removed from DS.$L$ depending on $\delta^*(u, v)$. This means that the bottleneck for a fast update time is the computation of $\delta^*(u, v)$:

**Lemma 4.1.** The procedure *DS*.Insert and *DS*.Delete run in $\varepsilon^{-O(d)} \cdot \tilde{O}(\log \Phi)$ time, plus the time needed for $\varepsilon^{-O(d)} \cdot \log \Phi$ many computations of $\delta^*(u, v)$.

Unfortunately, we don't know how to design a data structure that maintains these distances. To get around this, we first observe that it suffices to maintain an approximation for $\delta^*(u, v)$.

**Definition 4.2.** We say a value $\tilde{d}$ is an $\alpha$-approximation for a value $d$ if $d \leq \tilde{d} \leq \alpha \cdot d$. For any function $d(u, v)$ on pairs of points (we will take $d(u, v) = \delta^*(u, v)$ and $d(u, v) = \delta_{DS.L}(u, v)$), we say that $\tilde{d}(u, v)$ is a coarse $\alpha$-approximation for $d(u, v)$ if:

$$\tilde{d}(u, v) = \begin{cases} \text{if } d(u, v) \leq 2\delta(u, v) \text{ then} & d(u, v) \leq \tilde{d}(u, v) \leq \alpha \cdot d(u, v) \\ \text{if } d(u, v) > 2\delta(u, v) \text{ then} & \tilde{d}(u, v) \geq 2 \cdot \delta(u, v) \end{cases}$$

In other words, a coarse $\alpha$-approximation $\tilde{d}(u, v)$ for $\delta^*(u, v)$ lets us either detect when $\delta^*(u, v)$ is very large or provides an $\alpha$-approximation for $\delta^*(u, v)$. Throughout this section, we set $\alpha := 1 + \frac{\varepsilon}{3}$. As we show in Lemma 4.5, even if we used coarse $\alpha$-approximations $\tilde{\delta}^*(u, v)$ instead of the real distances $\delta^*(u, v)$ in the DS.Recompute procedure, our spanner $L$ still satisfies Invariants 2.7 and 2.8 after the DS.Insert and DS.Delete procedures.

**Augmenting the data structure with distance estimates.** We now show that we can modify our data structure to efficiently maintain coarse $\alpha$-approximations of $\delta^*(u, v)$ for every edge $(u, v)$ in the $\varepsilon$-net-tree DS.$S$. In fact, we maintain something stronger: we need to store several auxiliary values in order to maintain the coarse approximations. Let $\kappa \geq \frac{1024}{3}$ be a sufficiently large constant, and let $\varepsilon_{\text{small}} \leftarrow \frac{\varepsilon}{3 \cdot \kappa \cdot \log \Phi}$. Our data structure DS maintains (in addition to the points DS.$X$, the net-tree DS.$\mathcal{N}$, the $\varepsilon$-net-tree spanner DS.$S$, and the light spanner DS.$L$) the following objects:

- An $\varepsilon_{\text{small}}$-net tree spanner $DS.S_{\text{small}}$ for DS.$\mathcal{N}$
- For every scale $i \in [\log \Phi]$ and every edge $(u, v)$ in DS.$S_{\text{small}}$ at scale $i$, a value $DS.\tilde{\delta}^*(u, v)$ which is a coarse $(1 + \kappa \cdot i \cdot \varepsilon_{\text{small}})$-approximation for $\delta^*(u, v)$.

- For every scale $i \in [(\log \Phi) - 2]$ and every edge $(u, v)$ in $\texttt{DS}.S_{\text{small}}$ at scale $i$, a value $\textit{DS}.\tilde{\delta}_L(u, v)$ which is a $(1 + \kappa \cdot i \cdot \varepsilon_{\text{small}})$-approximation for $\delta_{\texttt{DS}.L}(u, v)$.

Observe that this data structure does indeed maintain a coarse $\frac{\varepsilon}{3}$-approximation for every edge in $\texttt{DS}.S$: this is because $\texttt{DS}.S \subset \texttt{DS}.S_{\text{small}}$, and $\kappa \cdot i \cdot \varepsilon_{\text{small}} < \frac{\varepsilon}{3}$ so the estimates $\texttt{DS}.\tilde{\delta}^*(u, v)$ are coarse $\frac{\varepsilon}{3}$-approximations for $\delta^*(u, v)$. The reason we maintain estimates for every edge in $\texttt{DS}.S_{\text{small}}$ and not just every edge in $\texttt{DS}.S$ is to control the distortion, which will accumulate by a $(1 + O(\varepsilon_{\text{small}}))$ factor at each scale. We highlight two differences between the estimates $\texttt{DS}.\tilde{\delta}^*(u, v)$ and $\texttt{DS}.\tilde{\delta}_L(u, v)$, for a scale-$i$ edge $(u, v)$. First, the value $\texttt{DS}.\tilde{\delta}^*(u, v)$ seeks to estimate $\delta^*(u, v)$, which is the distance between $u$ and $v$ in the subgraph $\texttt{DS}.L[i]$ which consists of all edges in $\texttt{DS}.L$ with scale strictly smaller than $i$. On the other hand, $\texttt{DS}.\tilde{\delta}_L(u, v)$ estimates distances in the graph $\texttt{DS}.L$; note that the shortest path between $u$ and $v$ in $\texttt{DS}.L$ may include edges in $\texttt{DS}.L$ at scale $i$ and $i + 1$, in addition to edges in $\texttt{DS}.L[i]$ (see Observation 4.6). The second key difference is that $\texttt{DS}.\tilde{\delta}^*(u, v)$ is only required to be a $(1 + \kappa \cdot i \cdot \varepsilon_{\text{small}})$-approximation of $\delta^*(u, v)$ when $\delta^*(u, v) \leq 2\delta(u, v)$, whereas $\texttt{DS}.\tilde{\delta}_L(u, v)$ is always a $(1 + \kappa \cdot i \cdot \varepsilon_{\text{small}})$-approximation of $\delta_{\texttt{DS}.L}(u, v)$. Why do we need these two types of estimates? Ultimately, our algorithm for Insert and Delete will only use the estimates $\texttt{DS}.\tilde{\delta}^*(u, v)$, to determine whether or not the edge $(u, v)$ should be added. But in order to compute $\texttt{DS}.\tilde{\delta}^*(u, v)$ for some pair $(u, v)$ at scale $i$, we will need to estimate $\delta_{\texttt{DS}.L}(u', v')$ for pairs $(u', v')$ at scale $\leq i - 3$.

**The fast insert/delete procedures.** Below, we give pseudocode for a modified DS.Recompute procedure called *DS*.RecomputeFast, which uses the estimates $\texttt{DS}.\tilde{\delta}^*(u, v)$ instead of the true distances $\delta^*(u, v)$. We define the procedures *DS*.InsertFast and *DS*.DeleteFast to be identical to DS.Insert and DS.Delete, except that they use DS.RecomputeFast instead of DS.Recompute, and they also maintain a $\varepsilon_{\text{small}}$-net-tree spanner $\texttt{DS}.S_{\text{small}}$ in addition to the $\varepsilon$-net-tree spanner $\texttt{DS}.S$. In Section 4.1, we describe the procedure DS.UpdateDistEstimates and prove that it correctly maintains estimates for $\delta^*(u, v)$ and $\delta_L(u, v)$. For now, we just state lemmas which summarize the guarantees of DS.UpdateDistEstimates (Lemmas 4.3 and 4.4). Assuming the

```
DS.RecomputeFast(x):
  for every scale i ← 0, 1, ..., log φ:
      DS.UpdateDistEstimates(x, i) ⟨⟨recomputes estimates DS.δ̃*(·,·) and DS.δ̃_L(·,·).⟩⟩
      for every scale-i edge (u, v) of DS.S with u, v ∈ B(x, 8 · 2^i):
          d̃ ← DS.δ̃*(u, v) ⟨⟨a ε/3-good estimate for δ*(u, v)⟩⟩
          if d̃ > (1 + ε) · δ(u, v):
              DS.L ← DS.L ∪ {(u, v)}
          else:
              DS.L ← DS.L \ {(u, v)}
```

lemmas hold, we prove the correctness of the fast insert/delete procedures (Lemma 4.5).

**Lemma 4.3.** *DS*.UpdateDistEstimates runs in time $(\varepsilon^{-1} \cdot \log \Phi)^{O(d)}$.

**Lemma 4.4.** Consider the execution of *DS*.InsertFast$(x)$ or *DS*.DeleteFast$(x)$. Let $i \in [\log \Phi]$ be a scale, and consider the data structure *DS* at the instant after the execution of the call *DS*.UpdateDistEstimates$(x, i)$ during the insertion/deletion process. Assume that every edge $(u, v)$ of *DS*.$S$ with scale strictly smaller than $i$ satisfies Invariants 2.7 and 2.8. Then:

- for every scale-$i$ edge $(u, v)$ in the $\varepsilon_{\text{small}}$-net-tree spanner *DS*.$S_{\text{small}}$, the value *DS*.$\tilde{\delta}^*(u, v)$ is a coarse $(1 + \kappa \cdot i \cdot \varepsilon_{\text{small}})$-approximation for $\delta^*(u, v)$.
- for every scale-$(i-2)$ edge $(u, v)$ in the $\varepsilon_{\text{small}}$-net-tree spanner *DS*.$S_{\text{small}}$, the value *DS*.$\tilde{\delta}_L(u, v)$ is a $(1+\kappa \cdot i \cdot \varepsilon_{\text{small}})$-approximation $\delta_{DS.L}(u, v)$.

**Lemma 4.5.** The procedure *DS*.InsertFast$(x)$ (resp. *DS*.DeleteFast$(x)$) correctly implements insertion (resp. deletion) of $x$. It runs in time $(\varepsilon^{-1} \cdot \log \Phi)^{O(d)}$.

Proof. The runtime bound follows from Lemmas 4.1 and 4.3. The proof of correctness is almost identical to that of Lemmas 3.1 and 3.2 except in one place: we need to argue that every scale-$i$ edge $(u, v)$ in DS.$S$ with both endpoints in $B(x, 4 \cdot 2^i)$ satisfies Invariants 2.7 and 2.8. For every such edge $(u, v)$, the algorithm checks if DS.$\tilde{\delta}^*(u, v) > (1 + \varepsilon)\delta(u, v)$ and includes $(u, v)$ in $L$ iff the inequality holds. At the instant the algorithm makes the check, we may assume (by induction) that every edge $(u', v')$

in $\texttt{DS}.S$ with scale strictly smaller than $i$ satisfies Invariants 2.7 and 2.8. Thus, by Lemma 4.4, the value of $\texttt{DS}.\tilde{\delta}(u,v)$ is a coarse $(1+\kappa \cdot i \cdot \varepsilon_{\text{small}})$-approximation for $\delta^*(u,v)$. As $i \in [\log \Phi]$, in particular we conclude that $\texttt{DS}.\tilde{\delta}(u,v)$ is a coarse $(1+\frac{\varepsilon}{3})$-approximation for $\delta^*(u,v)$. There are two cases. **Case 1: $\texttt{DS}.\tilde{\delta}(u,v) \leq (1+\varepsilon)\delta(u,v)$.** In this case, $(u,v)$ is not in $\texttt{DS}.L$. We claim that $\delta^*(u,v) \leq (1+\varepsilon)\delta(u,v)$, so that $(u,v)$ satisfies Invariant 2.7. Indeed, we have $\delta^*(u,v) < 2\delta(u,v)$; otherwise, the definition of coarse $\alpha$-approximation means that $\texttt{DS}.\tilde{\delta}(u,v) > 2\delta(u,v)$, which contradicts our assumption that $\texttt{DS}.\tilde{\delta}(u,v) \leq (1+\varepsilon)\delta(u,v)$. Thus, the definition of coarse $\alpha$-approximation implies $\delta^*(u,v) \leq \texttt{DS}.\tilde{\delta}(u,v)$ and so $\delta^*(u,v) \leq (1+\varepsilon)\delta(u,v)$ as desired. **Case 2: $\texttt{DS}.\tilde{\delta}(u,v) > (1+\varepsilon)\delta(u,v)$.** In this case, $(u,v)$ is added to $\texttt{DS}.L$. We claim that $\delta^*(u,v) > (1+\frac{\varepsilon}{3})\delta(u,v)$, so that $(u,v)$ satisfies Invariant 2.8. For contradiction, suppose that $\delta^*(u,v) \leq (1+\frac{\varepsilon}{3})\delta(u,v)$. The definition of coarse $(1+\frac{\varepsilon}{3})$-approximation implies $\texttt{DS}.\tilde{\delta}(u,v) \leq (1+\frac{\varepsilon}{3}) \cdot \delta^*(u,v)$. Combining these two inequalities, we conclude that $\texttt{DS}.\tilde{\delta}(u,v) \leq \left(1+\frac{\varepsilon}{3}\right)\left(1+\frac{\varepsilon}{3}\right)\delta(u,v) \leq (1+\varepsilon)\delta(u,v)$, contradicting the Case 2 assumption. □

Theorem 1.2 follows from Lemmas 4.5,2.9, and 2.10 (after rescaling $\varepsilon \leftarrow \varepsilon/3$).

## 4.1 The procedure UpdateDistEstimates

It remains to describe the UpdateDistEstimates$(x,i)$ procedure and prove Lemma 4.4. This procedure recomputes the distance estimates $\texttt{DS}.\tilde{\delta}^*(\cdot,\cdot)$ for the scale-$i$ edges of $\texttt{DS}.S_{\text{small}}$. It is described in pseudocode below, and summarized in text immediately after. The key subroutine is a procedure Estimate$(u,v,i)$ which estimates $\delta_{\texttt{DS}.L[i]}(u,v)$—that is, distances using edges of $\texttt{DS}.L$ with scale strictly smaller than $i$—under the assumption that the data structures stores accurate estimates $\texttt{DS}.\tilde{\delta}_L(u',v')$ for $\delta_{\texttt{DS}.L[i]}$ for every pair of $(u',v')$ at scale $\leq i-3$. To do this, the estimation procedure constructs a constant-size sketch graph $H$ and returns the distance between $u$ and $v$ in $H$. The sketch graph $H$ contains every scale-$(i-1)$ and scale-$(i-2)$ edge in $\texttt{DS}.L$ that is close to $u$ or $v$. Additionally, (roughly speaking) $H$ contains edges $(u',v')$ be-

DS.UpdateDistEstimates$(x, i)$:
  for every scale-$(i-2)$ edge $(u, v)$ of DS.$S_{\text{small}}$ with $u, v \in B(x, 4 \cdot 2^i)$:
    DS.$\tilde{\delta}_L(u, v) \leftarrow$ DS.Estimate$(u, v, i)$
  for every scale-$i$ edge $(u, v)$ of DS.$S_{\text{small}}$ with $u, v \in B(x, 4 \cdot 2^i)$:
    DS.$\tilde{\delta}^*(u, v) \leftarrow$ DS.Estimate$(u, v, i)$

DS.Estimate$(u, v, i)$:
  ⟨⟨*Returns an estimate of the distance* $\delta_{DS.L[i]}(u, v)$⟩⟩
  $i' \leftarrow \max\{0, \log(\varepsilon_{\text{small}} 2^i)\}$
  $H \leftarrow$ graph with vertex set $N_{i'} \cap B(x, 7 \cdot 2^i)$
  for every pair $(u', v')$ in $V(H)$ with $2^{i-3} \le \delta(u', v') < 2^{i-1}$:
    ⟨⟨*Add all edges with scale* $i-1$ *or* $i-2$⟩⟩
    if $(u', v') \in$ DS.$L$, then add edge $(u', v')$ to $H$ with weight $\delta(u', v')$
  for every pair $(u', v')$ in $V(H)$ with $\delta(u', v') < 2^{i-4}$:
    ⟨⟨*Estimate distances using scale* $\le i-3$ *edges*⟩⟩
    add edge $(u', v')$ to $H$ with weight DS.$\tilde{\delta}_L(u', v')$
  return $\delta_H(u, v)$

tween the endpoints of different large-scale edges[7], which are weighted using the estimates DS.$\tilde{\delta}_L(u', v')$, for pairs $(u', v')$ at scale $\le i-3$. These two types of edges are sufficient to preserve the shortest path between $u$ and $v$ in DS.$L[i]$. We first observe that DS.UpdateDistEstimates runs in $(\varepsilon^{-1} \cdot \log \Phi)^{O(d)}$ time.

of Lemma 4.3. Each call to DS.Estimate runs in time $(\varepsilon^{-1} \cdot \log \Phi)^{O(d)}$: indeed, it follows from the packing bound (Observation 2.1) that $H$ has at most $(\varepsilon^{-1} \cdot \log \Phi)^{O(d)}$ vertices, and it follows from Corollary 2.6 that these vertices can be found in $\varepsilon^{-O(d)}$ time. Overall, DS.UpdateDistEstimates makes $\varepsilon^{-O(d)} \cdot \log \Phi$ calls to DS.Estimate. □

We now prove Lemma 4.4, that is, correctness of the algorithm. Our proof is by induction on $i$. Given a scale $i$, we (inductively) assume that Lemma 4.4 holds for all scales $\le i-1$; in particular, we assume that for every edge $(u', v')$ of DS.$S_{\text{small}}$ with scale $\le i-3$, the value DS.$\tilde{\delta}_L(u', v')$ is a $(1 + \kappa \cdot (i-3) \cdot \varepsilon_{\text{small}})$-approximation for $\delta_L(u'v')$. We also assume that Invariants 2.7 and 2.8 hold for all edges of DS.$S$ with scale $\le i-1$. Throughout this section, we

[7] in practice, we also add edges between some net points which are not endpoints of a large-scale edge

write $\mathcal{N}$, $S$, $S_{\text{small}}$, and $L$ instead of $\texttt{DS}.\mathcal{N}$, $\texttt{DS}.S$, $\texttt{DS}.S_{\text{small}}$ and $\texttt{DS}.L$ (after the insertion/deletion procedure terminates). Our key claim is that $\texttt{DS}.\text{Estimate}(u, v, i)$ returns a coarse approximation of $\delta_{L[i]}(u, v)$. We begin with a simple observation that $\delta_L(u', v') = \delta_{L[i]}(u', v')$, for pairs of vertices $(u', v')$ at scale $\leq i - 2$.

**Observation 4.6.** Let $a$ and $b$ be two vertices with $\delta(a, b) \leq 2^j$ for some scale $j \leq i - 2$. We have that $\delta_L(a, b) < 2 \cdot \delta(a, b)$. We conclude that that the shortest path between $a$ and $b$ in $L$ uses edge with scale at most $j + 1$; that is, $\delta_L(a, b) = \delta_{L[j+2]}(u', v')$.

Proof. By assumption, our spanner $L$ satisfies Invariant 2.7 for every edge in $S$ with scale $\leq i-1$. We would like to apply Lemma 2.9 to conclude that $\delta_L(a, b) \leq (1 + 3\varepsilon)\delta(a, b) < 2\delta(a, b)$. However, we cannot do this directly, because Lemma 2.9 assume that Invariant 2.7 holds for all edges in $S$, not just those with scale $\leq i - 1$. This issue is easy to fix: simply consider the graph $L^+$ which is the union of $L$ and all scale-$j$ edges in $S$ with $j \geq i$. Clearly $L^+$ satisfies Lemma 2.9, so it is a $(1 + \varepsilon)$-spanner. This means that the shortest path between $a$ and $b$ in $L^+$ only uses edges with scale strictly smaller than $j + 2 \leq i$: an edge with scale $\geq i$ has length at least $2^{i-1}$, but we have $\delta_L(u, v)$ is strictly smaller than $2\delta(u, v) < 2 \cdot 2^j \leq 2^{i-1}$. We conclude that this path also appears in $L$. □

**Lemma 4.7.** Let $(u, v)$ be an edge of $S_{\text{small}}$ with scale $\leq i$, and $u, v \in B(x, 4 \cdot 2^i)$. Then $\texttt{DS}.\text{Estimate}(u, v, i)$ returns a coarse $(1+\kappa \cdot i \cdot \varepsilon_{\text{small}})$-approximation for $\delta_{L[i]}(u, v)$.

Proof. Let $H$ be the sketch graph constructed by $\texttt{DS}.\text{Estimate}(u, v, i)$.

**Lower bound.** First we prove $\delta_{L[i]}(u, v) \leq \delta_H(u, v)$. It suffices to show that, for any edge $(u', v')$ in $H$ with weight $w$, we have $\delta_{L[i]}(u, v) \leq w$. There are two types of edges that get added to $H$. In the first case, $(u', v')$ is an edge of $\texttt{DS}.L$ with scale either $i - 2$ or $i - 1$, and it belongs to $\texttt{DS}.L$; in this case, we have $w = \delta(u', v') = \delta_{L[i]}(u, v)$. In the second case, $(u', v')$ has scale at most $i - 3$, and $w = \texttt{DS}.\tilde{\delta}_L(u', v')$. By Observation 4.6 and induction hypothesis, we have $\delta_{L[i]}(u, v) = \delta_{\texttt{DS}.L}(u, v) \leq \texttt{DS}.\tilde{\delta}_L(u', v') = w$ as desired.

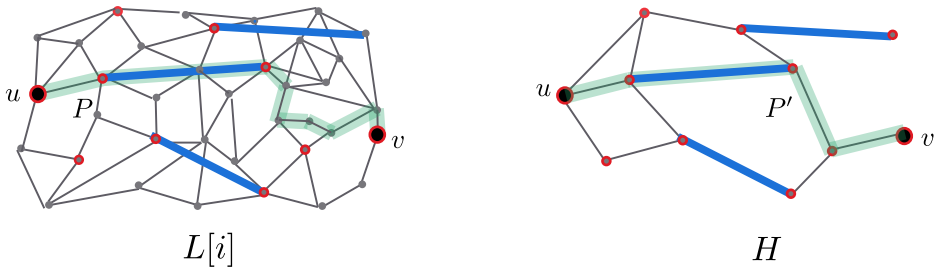



Figure 6: A depiction of the upper bound proof of Lemma 4.7. The graph $L[i]$, and the sketch graph $H$. The net points $N_{i'}$ are outlined in red, in both $L[i]$ and $H$. The "large-scale" edges of $L[i]$, with scale $(i-1)$ and $(i-2)$, are drawn in blue. A pair $(u,v)$ and the shortest path $P$ between them in $L[i]$ is marked in green, and the corresponding path $P'$ in $H$ is marked in green.

**Upper bound.** We now show that $\delta_H(u,v) \leq (1+(i\cdot\varepsilon_{\text{small}})$ if $\delta_{L[i]}(u,v) \leq 2\delta(u,v)$.[8] Refer to Figure 6. Consider the shortest path $P$ between $u$ and $v$ in $L[i]$. We argue that we can pull back $P$ onto $H$ with only small distortion. As $\delta_{L[i]}(u,v) \leq 2\delta(u,v)$, path $P$ only contains edges with both endpoints in $B(u, 2\cdot 2^i)$. By triangle inequality, path $P$ only contains edges with both endpoints in $B(x, 6\cdot 2^i)$. We divide $P$ into subpaths as follows. Treat $P$ as a directed path starting at $u$ and ending at $v$. Let $x_0 \leftarrow u$. For every $j > 0$, we define $x_j$ to be the farthest point along $P$ such that the subpath $P[x_{j-1} : x_j]$ has length smaller than $2^i/16$; if no such $x_j$ exists (because the edge following $x_{j-1}$ has length at least $2^i/16$) we set $x_j$ to be the vertex that comes immediately after $x_{j-1}$ in $P$. We define the subpath $P_j \coloneqq P[x_{j-1} : x_j]$ We write $P$ as the concatenation of subpaths

$$P = P_1 \circ P_2 \circ \ldots \circ P_k.$$

Observe that every subpath $P_j$ either consists of a single edge of $L[i]$ with scale $\geq i-3$, or it is a path in $L[i-3]$ (ie, it consists of edges of $L$ with scale $\leq i-4$). Moreover, there are at most $k = 64$ subpaths in this decomposition, as (by construction) any two consecutive subpaths together have length $\geq 2^i/16$, and $P$

[8] If $\delta_{L[i]}(u,v) > 2\delta(u,v)$, then the previous paragraph implies $\delta_H(u,v) \geq 2\delta(u,v)$, so we are done.

has length at most $2 \cdot \delta(u,v) \le 2 \cdot 2^i$. We now construct a path $P'$ in $H$ between $u$ and $v$, using path $P$. Intuitively, we will argue that each subpath $P_j$ can be pulled back to an edge in $H$; this forms the path $P'$. Let $i' = \max(0, \log(\varepsilon_{\text{small}} \cdot 2^i))$, and let $N_{i'} \in \texttt{DS}.\mathcal{N}$ be the net at scale $i'$. Observe $N_{i'} \cap B(x, 7 \cdot 2^i)$ are the vertices of $H$. For every point $x_j$, we define the point $x'_j$ to be the closest point in $N_{i'}$. By definition of net, we have $\delta(x_j, x'_j) \le \varepsilon_{\text{small}} 2^i < 2^i$, and so triangle inequality implies that $x'_j \in B(x, 7 \cdot 2^i)$. That is, each $x'_j$ is a vertex in the sketch graph $H$. Also note that $x_0 = x'_0 = u$ and $x_k = x'_k = v$: indeed, $u$ and $v$ are in $N_{i'}$, because $(u,v)$ is a scale-$i$ edge of $S_{\text{small}}$. Finally, we define $P'_j$ to be the shortest path in $H$ between $x'_{j-1}$ and $x'_j$. Define

$$P' = P'_1 \circ P'_2 \circ \ldots \circ P'_k.$$

Observe that $P'$ is a path in $H$ between $u$ and $v$. We prove below that for every subpath $P'_j$,

$$\delta_H(x'_{j-1}, x'_j) \le (1 + \kappa \cdot (i-3) \cdot \varepsilon_{\text{small}}) \cdot \text{len}(P_j) + 8 \cdot \varepsilon_{\text{small}} \cdot 2^i. \tag{1}$$

Summed over the 64 subpaths of $P$, Equation 1 implies that there is a path in $H$ between $u$ and $v$ with length at most $(1 + \kappa \cdot (i-3) \cdot \varepsilon_{\text{small}}) \delta_{L[i]}(u,v) + 512 \cdot \varepsilon_{\text{small}} 2^i$. As $\delta_{L[i]}(u,v) \ge \delta(u,v) \ge 2^i/2$ and $\kappa \ge \frac{1024}{3}$ is sufficiently large, we conclude $\delta_H(u,v) \le (1 + \kappa \cdot i \cdot \varepsilon_{\text{small}}) \delta_{L[i]}(u,v)$.

**Proof of Equation 1.** There are two cases. In the first case, suppose that $P_j$ consists of a single edge in $L[i]$ with scale $(i-1)$ or $(i-2)$. In this case, the endpoints $x_{j-1}$ and $x_j$ of $P_j$ are in the net $N_{i'}$, and so $x_{j-1} = x'_{j-1}$ and $x_j = x'_j$. The graph $H$ includes the edge $(x'_{j-1}, x'_j)$ with weight equal to $\text{len}(P_j)$, as desired. In the second case, suppose that that $P_j$ is either a scale-$(i-3)$ edge or a path in $L[i-3]$. First we show that $(x'_{j-1}, x'_j)$ is an edge in the $\varepsilon_{\text{small}}$-net-tree spanner $S_{\text{small}}$ with scale $\le i-3$. As $x'_{j-1}, x'_j \in N_{i'}$, the definition of $S_{\text{small}}$ implies that it suffices to show that $\delta(x'_{j-1}, x'_j) < 2^{i-3}$. To this end, note that if $P_j$ is a scale-$(i-3)$ edge we have $x'_{j-1} = x_{j-1}$ and $x'_j = x_j$, and so $\delta(x'_{j-1}, x'_j) = \delta(x_{j-1}, x_j) < 2^{i-3}$. Otherwise, suppose $P_j$ is a path in $P[i-3]$. Observe that the definition of $N_{i'}$ implies $\delta(x_j, x'_j) \le \varepsilon_{\text{small}} \cdot 2^i$, and similarly $\delta(x_{j-1}, x'_{j-1}) \le \varepsilon_{\text{small}} \cdot 2^i$. Triangle inequality implies

$$\delta(x'_{j-1}, x'_j) \le \delta(x_{j-1}, x_j) + 2\varepsilon_{\text{small}} \cdot 2^i < \frac{2^i}{16} + 2\varepsilon_{\text{small}} \cdot 2^i < 2^{i-3}.$$

Thus, the graph $H$ contains an edge $(x'_{j-1}, x'_j)$ with weight $\texttt{DS}.\tilde{\delta}_L(x'_{j-1}, x'_j)$. By induction hypothesis and Observation 4.6, the value $\texttt{DS}.\tilde{\delta}_L(x'_{j-1}, x'_j)$ is a $(1 + \kappa \cdot (i-3) \cdot \varepsilon_{\text{small}})$-approximation for $\delta_{L[i-3]}(x'_{j-1}, x'_j)$. Now we claim that

$$\delta_{L[i-3]}(x'_{j-1}, x'_j) \leq \text{len}(P_j) + 4\varepsilon_{\text{small}} \cdot 2^i. \tag{2}$$

Indeed, $\delta(x_j, x'_j) < 2^{i-5}$, so Observation 4.6 implies we have $\delta_{L[i-3]}(x_j, x'_j) \leq 2\varepsilon_{\text{small}} \cdot 2^i$. Similarly $\delta_{L[i-3]}(x_{j-1}, x'_{j-1}) \leq 2\varepsilon_{\text{small}} \cdot 2^i$. Triangle inequality proves Equation 2. Finally, we conclude $\delta_H(x'_{j-1}, x'_j) \leq (1 + \kappa \cdot (i-3) \cdot \varepsilon_{\text{small}}) \cdot (\text{len}(P_j + 4\varepsilon_{\text{small}} \cdot 2^i) < (1 + \kappa \cdot (i-3) \cdot \varepsilon_{\text{small}}) \cdot \text{len}(P_j) + 8\varepsilon_{\text{small}} \cdot 2^i$. This proves Equation 1, and completes the proof of Lemma 4.7. □

We are now ready to prove Lemma 4.4.

Proof of Lemma 4.4. Let $S^{\text{old}}_{\text{small}}$ and $L^{\text{old}}$ denote $\texttt{DS}.S_{\text{small}}$ and $\texttt{DS}.L$ (respectively) before the insertion/deletion process. Let $(u, v)$ be a scale-$i$ edge of $S_{\text{small}}$. We claim that value $\texttt{DS}.\tilde{\delta}^*(u, v)$ is a coarse $(1 + \kappa \cdot i \cdot \varepsilon_{\text{small}})$-approximation of $\delta^*(u, v)$. Indeed, if $u, v \in B(x, 4 \cdot 2^i)$ then Lemma 4.7 implies that the data structure stores a coarse approximation. On the other hand, if one endpoint is not in $B(x, 4 \cdot 2^i)$, then Lemma 3.3 implies[9] that $(u, v)$ is an edge in $S^{\text{old}}_{\text{small}}$, and that either $\delta^*(u, v) = \delta_{L^{\text{old}}[i]}(u, v)$ or both $\delta^*(u, v)$ and $\delta_{L^{\text{old}}[i]}(u, v)$ are larger than $2\delta(u, v)$; thus, any coarse $\alpha$-approximation for $\delta_{L^{\text{old}}[i]}(u, v)$ is a coarse $\alpha$-approximation for $\delta_{L[i]}(u, v) = \delta^*(u, v)$. By assumption, before insertion/deletion we had stored a value $\texttt{DS}.\tilde{\delta}^*(u, v)$ which is a $(1 + \kappa \cdot i \cdot \varepsilon_{\text{small}})$-approximation for $\delta_{L[i]}(u, v)$, so we are done. Let $(u, v)$ be a scale-$(i-2)$ edge of $S_{\text{small}}$. By an identical argument, $\texttt{DS}.\tilde{\delta}_L(u, v)$ is a coarse $(1 + \kappa \cdot i \cdot \varepsilon_{\text{small}})$-approximation of $\delta_L(u, v)$. By Observation 4.6, we have $\delta_L(u, v) < 2\delta(u, v)$, and so $\texttt{DS}.\tilde{\delta}_L(u, v)$ is a (non-coarse) $(1 + \kappa \cdot i \cdot \varepsilon_{\text{small}})$-approximation of $\delta_L(u, v)$. □

**Remark 4.8** (Coping with an unknown aspect ratio.)**.** While we have written our data structure as though the aspect ratio $\Phi$ is known in advance, it is possible to remove this assumption by following [GGN04]. Rather than explicitly maintaining the set of

[9] Note that Lemma 3.3 is stated with DS.Insert and DS.Delete instead of DS.InsertFast and DS.DeleteFast, and it refers to edges of $\texttt{DS}.S$ instead of $\texttt{DS}.S_{\text{small}}$. But the statement still holds even with these changes.

points in each net $N_i$ for every scale $i \in [\log \Phi]$, [GGN04] implicitly maintain a net for every integer scale $i \in \mathbb{Z}$. Specifically, they maintain a value $i_{\max}$ (such that for all $i \geq i_{\max}$, the net $N_i$ consists of a single point) and a value $i_{\min}$ (such that for all $i \leq i_{\min}$, the net $N_i$ consists of the entire point set $X$) with $i_{\max} - i_{\min} = O(\log \Phi)$; for scales $i_{\min} \leq i \leq i_{\max}$, for each point $x$ in $N_i$, they store a list of points in $N_i$ within distance $O(2^i)$ of $x$. [GGN04] show that this implicit net tree can be maintained efficiently. In our algorithms for the dynamic spanner, we could remove the assumption that we know $\Phi$ by using the implicit net tree, and replacing the for-loop in *DS*.Recompute with a loop from $i \leftarrow i_{\min}$ up to $i_{\max}$. We implicitly maintain values $DS.\tilde{\delta}^*(u,v)$ and $DS.\tilde{\delta}_L(u,v)$ for every edge $(u,v)$ and every scale $i$ (for scales $i > i_{\max}$ there is only a single point in the net so this is trivial, and for scale $i < i_{\min}$ there are no scale-$i$ edges).

# 5 Conclusion and Open Questions

We have designed the first dynamic light spanner for point sets in Euclidean and doubling metrics, with update time $(\log \Phi)^{O(d)}$. Our work leaves open two interesting open questions. First, is it possible to maintain a dynamic light spanner with recourse or time bounds independent of the aspect ratio $\Phi$ and instead depends only on the number of points $n = |X|$? Second, can we obtain a runtime bound where the exponent of $\log \Phi$ (or $\log n$) is independent of $d$? Ideally, we would like to handle updates in time $\varepsilon^{-O(d)} \cdot \log n$, which would match the best runtime for dynamic sparse spanners.

# A LSO-Based Spanners Are Not Light

Roughly speaking, given a metric space $(X, \delta)$, an $(\tau, \varepsilon)$-LSO is a collection $\Sigma$ of $\tau$ orderings over $X$ such that every pair of points $x, y \in X$ is almost consequence in one of the orderings (see Definition A.1). Chan et al. [CHJ20] constructed $(\tilde{O}(\varepsilon)^{-d}, \varepsilon)$-LSO for the Euclidean space $\mathbb{R}^d$ (see also [GH24]). Later, Filtser and Le [FL22] constructed $(\varepsilon^{-O(d)}, \varepsilon)$-LSO for metrics with doubling dimension $d$. Given a $(\tau, \varepsilon)$-LSO, one can construct a spanner by

connecting any two points which are consecutive in some ordering [CHJ20]. The result is a $(1+O(\varepsilon))$-spanner with $(n-1)\cdot\tau$ edges. Chan et al. showed that it is possible to efficiently maintain the LSO for $\mathbb{R}^d$, and thus it become very easy to dynamically maintain a sparse spanner. Later, Le and La [LL25] constructed a dynamic version of the LSO for doubling metrics, thus allowing us to easily maintain dynamic spanner. Unfortunately, the resulting spanner is by no means light. Indeed, in Lemma A.2 we show that it has logarithmic lightness (even for $\mathbb{R}^1$). We begin with a formal definition of an LSO (even though the proof of Lemma A.2 will go though also using slightly weaker definition presented above).

**Definition A.1** ($(\tau,\rho)$-LSO)**.** Given a metric space $(X, d_X)$, we say that a collection $\Sigma$ of orderings is a $(\tau,\rho)$-LSO if $|\Sigma| \leq \tau$, and for every $x, y \in X$, there is a linear ordering $\sigma \in \Sigma$ such that (w.l.o.g.[10]) $x \preceq_\sigma y$ and the points between $x$ and $y$ w.r.t. $\sigma$ could be partitioned into two consecutive intervals $I_x, I_y$ where $I_x \subseteq B_X(x, \rho \cdot d_X(x,y))$ and $I_y \subseteq B_X(y, \rho \cdot d_X(x,y))$. $\rho$ is called the stretch parameter.

In Lemma A.2 below we prove that LSO-based spanner has logarithmic lightness even for the path graph (which is Euclidean, and has constant doubling dimension).

**Lemma A.2.** Consider a $(\tau,\varepsilon)$-LSO for the set of points $\left(v_1 = 1, v_2 = 2, \ldots, v_{2^\Phi} = 2^\Phi\right)$, for some $\varepsilon \in (0, \frac{1}{6})$. Let $H_{\mathrm{LSO}}$ be the $(1 + O(\varepsilon))$-spanner created by connecting every two points that are consecutive in some ordering. Then $w(H_{\mathrm{LSO}}) = \Omega(\Phi \cdot 2^\Phi)$. In particular, $H_{\mathrm{LSO}}$ has logarithmic lightness.

Proof. For $i \in [0, \Phi]$, let $N_i = \left\{v_j \mid j \mod 2^i = 0\right\}$ be a hierarchical net. For any pair of consecutive net points in $N_i$, $x = v_{q\cdot 2^i}$ and $y = v_{(q+1)\cdot 2^i}$ there is some ordering $\sigma$ such that $x$ and $y$ are almost consecutive in $\sigma$. In particular, there are points $x', y'$ which are consecutive along $\sigma$ such that $\delta(x,x') \leq \varepsilon \cdot \delta(x,y) = \varepsilon \cdot 2^i$, and $\delta(y,y') \leq \varepsilon \cdot 2^i$. Denote this edge by $e_{i,q}$. $e_{i,q}$ is part of the spanner resulting from the LSO, and $w(e_{i,q}) \in [1-2\varepsilon, 1+2\varepsilon] \cdot 2^i$. We argue that for every $(i,q) \neq (i',q')$ it holds that $e_{i,q} \neq e_{i',q'}$. For the sake of contradiction, suppose otherwise. We continue by case analysis:

[10] That is either $x \preceq_\sigma y$ or $y \preceq_\sigma x$, and the guarantee holds w.r.t. all the points between $x$ and $y$ in the order $\sigma$.

- $i \neq i'$. Suppose w.l.o.g. that $i < i'$. Then $w(e_{i,q}) \leq (1+2\varepsilon) \cdot 2^i < (1-2\varepsilon) \cdot 2^{i'} \leq w(e_{i',q'})$, a contradiction.
- $i = i'$. Suppose w.l.o.g. that $q < q'$. The edge $e_{i,q}$ has its left endpoint at distance at most $\varepsilon \cdot 2^i$ from $v_{q \cdot 2^i}$. The edge $e_{i',q'}$ has its right endpoint at distance at most $\varepsilon \cdot 2^i$ from $v_{(q'+1) \cdot 2^i}$. It follows that

$$\begin{aligned} w(e_{i,q}) &\geq \left|(q'+1) \cdot 2^i - q \cdot 2^i\right| - 2 \cdot \varepsilon \cdot 2^i \\ &\geq 2 \cdot 2^i - 2 \cdot \varepsilon \cdot 2^i = (1-\varepsilon) \cdot 2^{i+1} > (1+2\varepsilon) \cdot 2^i \,, \end{aligned}$$

  a contradiction. □

# References


[AA92] Noga Alon and Yossi Azar. On-line steiner trees in the euclidean plane. In David Avis, editor, Proceedings of the Eighth Annual Symposium on Computational Geometry, Berlin, Germany, June 10-12, 1992, pages 337–343. ACM, 1992.

[AAA+06] Noga Alon, Baruch Awerbuch, Yossi Azar, Niv Buchbinder, and Joseph Naor. A general approach to online network optimization problems. ACM Trans. Algorithms, 2(4):640–660, 2006.

[AdBG08] Mohammad Ali Abam, Mark de Berg, and Joachim Gudmundsson. A simple and efficient kinetic spanner. In Proceedings of the twenty-fourth annual symposium on Computational geometry, pages 306–310, 2008.

[ADD+93] Ingo Althöfer, Gautam Das, David Dobkin, Deborah Joseph, and José Soares. On sparse spanners of weighted graphs. Discrete & Computational Geometry, 9(1):81–100, 1993.

[ADM+95] Sunil Arya, Gautam Das, David M Mount, Jeffrey S Salowe, and Michiel Smid. Euclidean spanners: short, thin, and lanky. In Proceedings of the twenty-seventh annual ACM symposium on Theory of computing, pages 489–498, 1995.

[AFI05] Giorgio Ausiello, Paolo G Franciosa, and Giuseppe F Italiano. Small stretch spanners on dynamic graphs. In European Symposium on Algorithms, pages 532–543. Springer, 2005.

[AMS99] Sunil Arya, David M Mount, and Michiel Smid. Dynamic algorithms for geometric spanners of small diameter: Randomized solutions. Computational Geometry, 13(2):91–107, 1999.

[BC97] Piotr Berman and Chris Coulston. On-line algorithms for steiner tree problems (extended abstract). In Proceedings of the Twenty-Ninth Annual ACM Symposium on the Theory of Computing, El Paso, Texas, USA, May 4-6, 1997, pages 344–353. ACM, 1997.

[BFH21] Aaron Bernstein, Sebastian Forster, and Monika Henzinger. A deamortization approach for dynamic spanner and dynamic maximal matching. ACM Transactions on Algorithms (TALG), 17(4):1–51, 2021.

[BFKT24] Sujoy Bhore, Arnold Filtser, Hadi Khodabandeh, and Csaba D. Tóth. Online spanners in metric spaces. SIAM J. Discret. Math., 38(1):1030–1056, 2024.

[BGM04] Prosenjit Bose, Joachim Gudmundsson, and Pat Morin. Ordered theta graphs. Computational Geometry, 28(1):11–18, 2004.

[BK16] Greg Bodwin and Sebastian Krinninger. Fully dynamic spanners with worst-case update time. In 24th Annual European Symposium on Algorithms (ESA 2016), pages 17–1. Schloss Dagstuhl–Leibniz-Zentrum für Informatik, 2016.

[BKK+25] Sujoy Bhore, Balázs Keszegh, Andrey Kupavskii, Hung Le, Alexandre Louvet, Dömötör Pálvölgyi, and Csaba D. Tóth. Spanners in planar domains via steiner spanners and non-steiner tree covers. In Proceedings of the 2025 Annual ACM-SIAM Symposium on Discrete Algorithms (SODA), pages 4292–4326. SIAM, 2025.

[BKS12] Surender Baswana, Sumeet Khurana, and Soumojit Sarkar. Fully dynamic randomized algorithms for graph spanners. ACM Transactions on Algorithms (TALG), 8(4):1–51, 2012.

[BLW19] Glencora Borradaile, Hung Le, and Christian Wulff-Nilsen. Greedy spanners are optimal in doubling metrics. In Proceedings of the Thirtieth Annual ACM-SIAM Symposium on Discrete Algorithms, pages 2371–2379. SIAM, 2019.

[BLWN17] Glencora Borradaile, Hung Le, and Christian Wulff-Nilsen. Minor-free graphs have light spanners. In 2017 IEEE 58th Annual Symposium on Foundations of Computer Science (FOCS), pages 767–778. IEEE, 2017.

[BM25] Sujoy Bhore and Lazar Milenkovic. Light spanners with small hop-diameter. In 52nd International Colloquium on Automata, Languages, and Programming, ICALP 2025, Aarhus, Denmark, July 8-11, 2025, volume 334 of LIPIcs, pages 30:1–30:16. Schloss Dagstuhl - Leibniz-Zentrum für Informatik, 2025.

[Bod24] Greg Bodwin. An alternate proof of near-optimal light spanners. In Merav Parter and Seth Pettie, editors, 2024 Symposium on Simplicity in Algorithms, SOSA 2024, Alexandria, VA, USA, January 8-10, 2024, pages 39–55. SIAM, 2024.

[BT22] Sujoy Bhore and Csaba D. Tóth. Euclidean steiner spanners: Light and sparse. SIAM J. Discret. Math., 36(3):2411–2444, 2022.

[BT25] Sujoy Bhore and Csaba D. Tóth. Online euclidean spanners. ACM Trans. Algorithms, 21(1):5:1–5:22, 2025.

[CAFKL20] Vincent Cohen-Addad, Arnold Filtser, Philip N Klein, and Hung Le. On light spanners, low-treewidth embeddings and efficient traversing in minor-free graphs. In 2020 IEEE 61st Annual Symposium on

Foundations of Computer Science (FOCS), pages 589–600. IEEE, 2020.

[CCL+25] Hsien-Chih Chang, Jonathan Conroy, Hung Le, Shay Solomon, and Cuong Than. Light tree covers, routing, and path-reporting oracles via spanning tree covers in doubling graphs. In STOC'25—Proceedings of the 57th Annual ACM Symposium on Theory of Computing, pages 2257–2268. ACM, 2025.

[CG06] Richard Cole and Lee-Ad Gottlieb. Searching dynamic point sets in spaces with bounded doubling dimension. In Proceedings of the thirty-eighth annual ACM symposium on Theory of computing, pages 574–583, 2006.

[CG09] T-H Hubert Chan and Anupam Gupta. Small hop-diameter sparse spanners for doubling metrics. Discrete & Computational Geometry, 41(1):28–44, 2009.

[CHJ20] Timothy M Chan, Sariel Har-Peled, and Mitchell Jones. On locality-sensitive orderings and their applications. SIAM Journal on Computing, 49(3):583–600, 2020.

[CW18] Shiri Chechik and Christian Wulff-Nilsen. Near-optimal light spanners. ACM Transactions on Algorithms (TALG), 14(3):1–15, 2018.

[EK24] David Eppstein and Hadi Khodabandeh. Maintaining light spanners via minimal updates. In Proceedings of the 36th Canadian Conference on Computational Geometry (CCCG), pages 1–15, 2024.

[Elk11] Michael Elkin. Streaming and fully dynamic centralized algorithms for constructing and maintaining sparse spanners. ACM Transactions on Algorithms (TALG), 7(2):1–17, 2011.

[ENS15] Michael Elkin, Ofer Neiman, and Shay Solomon. Light spanners. SIAM J. Discret. Math., 29(3):1312–1321, 2015.

[ES15] Michael Elkin and Shay Solomon. Steiner shallow-light trees are exponentially lighter than spanning ones. SIAM J. Comput., 44(4):996–1025, 2015.

[ES16] Michael Elkin and Shay Solomon. Fast constructions of lightweight spanners for general graphs. ACM Trans. Algorithms, 12(3):29:1–29:21, 2016.

[FG19] Sebastian Forster and Gramoz Goranci. Dynamic low-stretch trees via dynamic low-diameter decompositions. In Proceedings of the 51st Annual ACM SIGACT Symposium on Theory of Computing, pages 377–388, 2019.

[FL22] Arnold Filtser and Hung Le. Locality-sensitive orderings and applications to reliable spanners. In Proceedings of the 54th Annual ACM SIGACT Symposium on Theory of Computing, pages 1066–1079, 2022.

[FN22] Arnold Filtser and Ofer Neiman. Light spanners for high dimensional norms via stochastic decompositions. Algorithmica, 84(10):2987–3007, 2022.

[FS20] Arnold Filtser and Shay Solomon. The greedy spanner is existentially optimal. SIAM J. Comput., 49(2):429–447, 2020.

[GGN04] Jie Gao, Leonidas J Guibas, and An Nguyen. Deformable spanners and applications. In Proceedings of the twentieth annual symposium on Computational geometry, pages 190–199, 2004.

[GH24] Zhimeng Gao and Sariel Har-Peled. Near optimal locality sensitive orderings in euclidean space. In 40th International Symposium on Computational Geometry, SoCG 2024, Athens, Greece, June 11-14, 2024, volume 293 of LIPIcs, pages 60:1–60:14. Schloss Dagstuhl - Leibniz-Zentrum für Informatik, 2024.

[Got15] Lee-Ad Gottlieb. A light metric spanner. In 2015 IEEE 56th Annual Symposium on Foundations of Computer Science, pages 759–772. IEEE, 2015.

[GR08a] Lee-Ad Gottlieb and Liam Roditty. Improved algorithms for fully dynamic geometric spanners and geometric routing. In Proceedings of the Nineteenth Annual ACM-SIAM Symposium on Discrete Algorithms, SODA 2008, San Francisco, California, USA, January 20-22, 2008, pages 591–600. SIAM, 2008.

[GR08b] Lee-Ad Gottlieb and Liam Roditty. An optimal dynamic spanner for doubling metric spaces. In European Symposium on Algorithms, pages 478–489. Springer, 2008.

[GS02] Michelangelo Grigni and Papa Sissokho. Light spanners and approximate tsp in weighted graphs with forbidden minors. In SODA, volume 2, pages 852–857, 2002.

[HIS13] Sariel Har-Peled, Piotr Indyk, and Anastasios Sidiropoulos. Euclidean spanners in high dimensions. In Sanjeev Khanna, editor, Proceedings of the Twenty-Fourth Annual ACM-SIAM Symposium on Discrete Algorithms, SODA 2013, New Orleans, Louisiana, USA, January 6-8, 2013, pages 804–809. SIAM, 2013.

[HLP17] MohammadTaghi Hajiaghayi, Vahid Liaghat, and Debmalya Panigrahi. Online node-weighted steiner forest and extensions via disk paintings. SIAM J. Comput., 46(3):911–935, 2017.

[HPM06] Sariel Har-Peled and Manor Mendel. Fast construction of nets in low-dimensional metrics and their applications. SIAM J. Comput., 35(5):1148–1184, 2006.

[KL04] Robert Krauthgamer and James R. Lee. Navigating nets: simple algorithms for proximity search. In Proceedings of the Fifteenth Annual ACM-SIAM Symposium on Discrete Algorithms, pages 798–807. ACM, New York, 2004.

[LL25] An La and Hung Le. Dynamic locality sensitive orderings in doubling metrics. In Proceedings of the

57th Annual ACM Symposium on Theory of Computing, pages 1178–1189, 2025.

[LS23] Hung Le and Shay Solomon. A unified framework for light spanners. In Proceedings of the 55th Annual ACM Symposium on Theory of Computing, STOC 2023, Orlando, FL, USA, June 20-23, 2023, pages 295–308. ACM, 2023.

[LS25] Hung Le and Shay Solomon. Truly optimal euclidean spanners. SIAM J. Comput., 54(4):S19–135, 2025.

[NPS11] Joseph Naor, Debmalya Panigrahi, and Mohit Singh. Online node-weighted steiner tree and related problems. In IEEE 52nd Annual Symposium on Foundations of Computer Science, (FOCS), pages 210–219. IEEE Computer Society, 2011.

[NS07] Giri Narasimhan and Michiel Smid. Geometric spanner networks. Cambridge University Press, Cambridge, 2007.

[PS89] David Peleg and Alejandro A Schäffer. Graph spanners. Journal of graph theory, 13(1):99–116, 1989.

[Rod12] Liam Roditty. Fully dynamic geometric spanners. Algorithmica, 62(3):1073–1087, 2012.

[RS98] Satish B Rao and Warren D Smith. Approximating geometrical graphs via “spanners” and “banyans”. In Proceedings of the thirtieth annual ACM symposium on Theory of computing, pages 540–550, 1998.

[Smi09] Michiel HM Smid. The weak gap property in metric spaces of bounded doubling dimension. Efficient Algorithms, 5760:275–289, 2009.